

QueryDSL세팅하기

: IntelliJ IDE + Maven

QueryDSL 은 Java 기반의 타입 안전 쿼리 생성기로 Spring Data JPA 와 함께 사용하면, 복잡한 쿼리를 더 안전하고 간결하게 작성할 수 있다.

QueryDSL 의 주요 특징

1. **타입 안전성** : QueryDS 은 쿼리를 작성할 때 컴파일러가 오류를 체크하므로 런타임 오류를 줄일 수 있다.
2. **유연한 쿼리 생성** : QueryDSL 을 사용하면 동적으로 쿼리를 생성할 수 있어 조건에 따라 쿼리의 일부를 조합하거나 변경하는 것이 용이하다.
3. **JPA 와의 통합** : Spring Data JPA 와 통합하여 JPA 의 엔티티를 기반으로 쿼리를 작성할 수 있다.
4. **다양한 데이터베이스 지원** : QueryDSL 은 JPA 외에도 SQL, MongoDB, Hibernate 등 다양한 데이터베이스와의 연동을 지원한다

사용법

- 1) 의존성 추가
- 2) Q클래스 생성
- 3) 쿼리작성

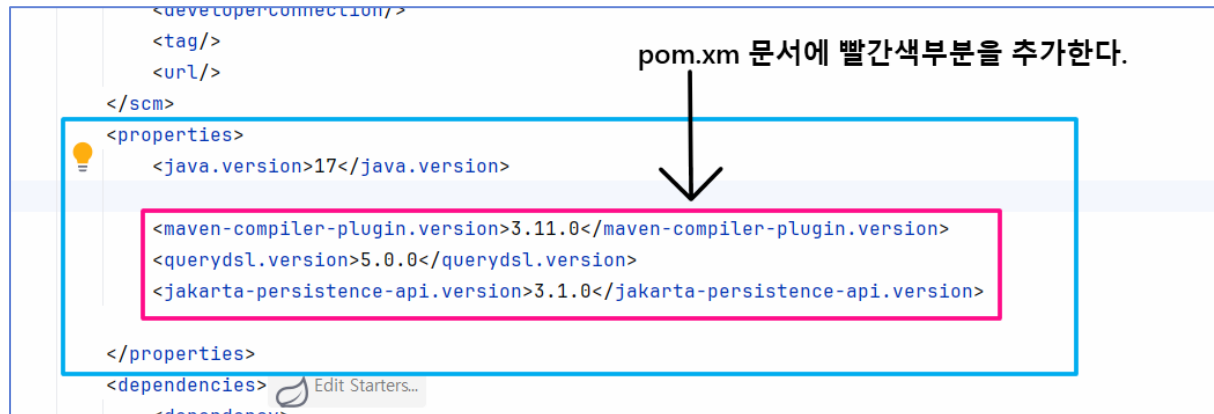
- pom.xml문서에 의존성 추가한다.

: <properties>안에 아래의 코드를 추가한다.

```

<maven-compiler-plugin.version>3.11.0</maven-compiler-plugin.version>
<querydsl.version>5.0.0</querydsl.version>
<jakarta-persistence-api.version>3.1.0</jakarta-persistence-api.version>

```



<dependencies> 안에 아래의 태그를 추가한다.

```

<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-apt</artifactId>
  <version>${querydsl.version}</version>
  <classifier>jakarta</classifier>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-jpa</artifactId>
  <classifier>jakarta</classifier>
  <version>${querydsl.version}</version>
</dependency>

```

```

2      <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
37      <dependencies>
100     <dependency>
102         <artifactId>jakarta.servlet.jsp.jstl</artifactId>
103     </dependency>
104
105     <dependency>
106         <groupId>com.querydsl</groupId>
107         <artifactId>querydsl-apt</artifactId>
108         <version>${querydsl.version}</version>
109         <classifier>jakarta</classifier>
110         <scope>provided</scope>
111     </dependency>
112     <dependency>
113         <groupId>com.querydsl</groupId>
114         <artifactId>querydsl-jpa</artifactId>
115         <classifier>jakarta</classifier>
116         <version>${querydsl.version}</version>
117     </dependency>
118
119 </dependencies>
120
121 <build>

```

pom.xml문서 아래쪽 plugin 추가

```

<!-- QueryDSL세팅 -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>${maven-compiler-plugin.version}</version>

    <configuration>
        <source>${java.version}</source>
        <target>${java.version}</target>
        <encoding>UTF-8</encoding>
        <annotationProcessorPaths>
            <path>
                <groupId>com.querydsl</groupId>
                <artifactId>querydsl-apt</artifactId>
                <version>${querydsl.version}</version>
                <classifier>jakarta</classifier>
            </path>
        </annotationProcessorPaths>
    </configuration>
</plugin>

```

```

        </path>
        <path>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
            <version>3.1.0</version>
        </path>
    </annotationProcessorPaths>
</configuration>
</plugin>

<!-- QueryDSL세팅끝 -->

```

```

84     <build>
85         <plugins>
109         </plugin>
110
111         <plugin>
112             <groupId>org.apache.maven.plugins</groupId>
113             <artifactId>maven-compiler-plugin</artifactId>
114             <version>${maven-compiler-plugin.version}</version> <configuration>
115                 <annotationProcessorPaths>
116                     <path>
117                         <groupId>org.projectlombok</groupId>
118                         <artifactId>lombok</artifactId>
119                         <version>${lombok.version}</version> </path>
120                     <path>
121                         <groupId>com.querydsl</groupId>
122                         <artifactId>querydsl-apt</artifactId>
123                         <version>${querydsl.version}</version>
124                         <classifier>jakarta</classifier>
125                     </path>
126                     <path>
127                         <groupId>jakarta.persistence</groupId>
128                         <artifactId>jakarta.persistence-api</artifactId>
129                         <version>${jakarta-persistence-api.version}</version>
130                     </path>
131                 </annotationProcessorPaths>
132                 <generatedSourcesDirectory>${project.build.directory}/generated-sources/annotations</generatedSourcesDirectory>
133             </configuration>
134         </plugin>
135     </plugins>
136 </build>
137

```

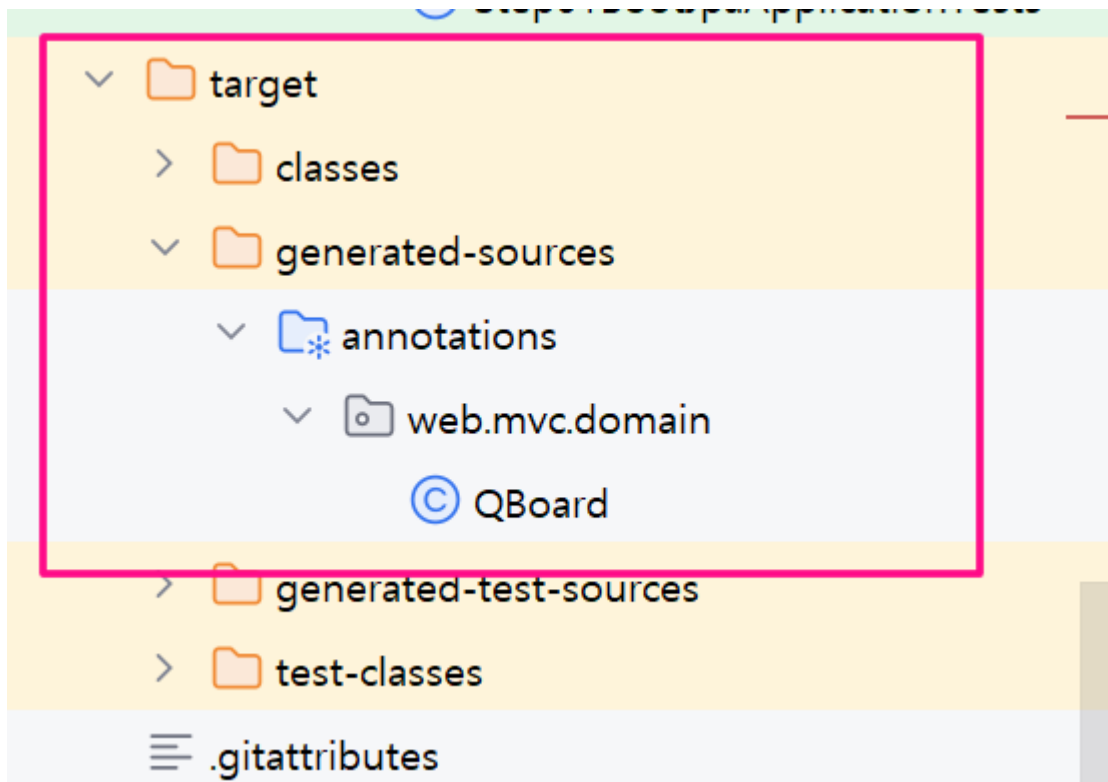
annotationProcessorPaths
를 설정하면
기본적으로
target/generated-sources/annotations 폴더에
Q파일 생성된다.
Maven Compiler Plugin의 기본동작

target/generated-sources/annotations 디렉터리에서

우클릭 → Mark Directory as → Generated Sources Root

- 설정 완료 후 엔티티 클래스에 대해 QueryDSL이 자동으로 생성하는 Q클래스 확인

프로젝트의 target > generated-sources > annotatios> 안에 Q 파일을 확인한다.



- Repository에서 QueryDSL을 사용하여 쿼리를 작성 할 수 있다.

JPAQueryFactory는 QueryDSL에서 제공하는 유틸리티 클래스로, JPA를 사용하여 쿼리를 작성하는 데 도움을 준다.

JPAQueryFactory 역할 및 기능

1. **쿼리 빌더 생성** : JPAQueryFactory 쿼리 객체를 쉽게 만들고 실행할 수 있다.
2. **타입 안전성 보장** : JPAQueryFactory 를 사용하면 QueryDSL 의 Q 타입을 활용하여 컴파일 타임에 쿼리의 유효성을 체크할 수 있다. 즉, 잘못된 속성명이나 타입을 사용할 경우 컴파일 에러로 알려주므로 안정성이 높아진다.
3. **엔티티 매니저 통합** : JPAQueryFactory 는 JPA 의 EntityManager 와 결합되어, 데이터베이스와의 상호작용을 간편하게 처리할 수 있다. EntityManager 를 주입하여 사용한다.

4. **동적 쿼리 작성** : 다양한 조건을 바탕으로 동적으로 쿼리를 생성할 수 있다. 예를 들어, 특정 조건에 따라 where 절에 조건을 추가하는 등의 유연한 쿼리 구성이 가능하다.

아래와 같이 **JPAQueryFactory를 Bean으로 등록한다.**

JPAQueryFactory를 bean으로 등록할 때 생성자를 통해 EntityManager를 전달한다.

EntityManager는 @PersistenceContext 애노테이션을 이용하여 주입 받는다.

@PersistenceContext는 JPA의 EntityManager를 주입 받는 데 사용된다. 이를 통해 데이터베이스와의 상호작용을 간편하게 처리할 수 있다.

```
1 package web.mvc.config;
2
3 > import ...
4
5 @Configuration //환경설정을 돕는 클래스 new *
6 public class AppConfig {
7
8     @PersistenceContext 1 usage
9     private EntityManager entityManager;
10
11     @Bean new *
12     public JPAQueryFactory getJPAQueryFactory(){
13         return new JPAQueryFactory(entityManager);
14     }
15 }
16
```

```

////////////////////////////////////
@Autowired
private JPAQueryFactory jpaFactory;

@Test
@DisplayName("QueryDSL테스트")
public void queryDSL(){
    QBoard board = QBoard.board;
    List<Board> list = jpaFactory
        .selectFrom(board)
        .fetch();
    list.forEach(board1 -> System.out.println(board1));
}

```

실행취리

Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
 Hibernate: select b1_0.bno,b1_0.content,b1_0.insert_date,b1_0.title,b1_0.update_date,b1_0.writer from board b1_0
 2024-11-03T13:15:18.576+09:00 INFO 15076 --- [sten01 boot.JPA] [ionShutdownHook] i.LocalContainerEntityManagerFactoryBean

```

@Test
@DisplayName("QueryDSL 조건 테스트")
public void queryDSL1(){
    QBoard board = QBoard.board;
    List<Board> list = jpaFactory
        .selectFrom(board)
        .where(board.bno.gt( right: 50L).or(board.title.eq( right: "제목1"))).fetch();
    list.forEach(board1 -> System.out.println(board1));
}

```

실행취리

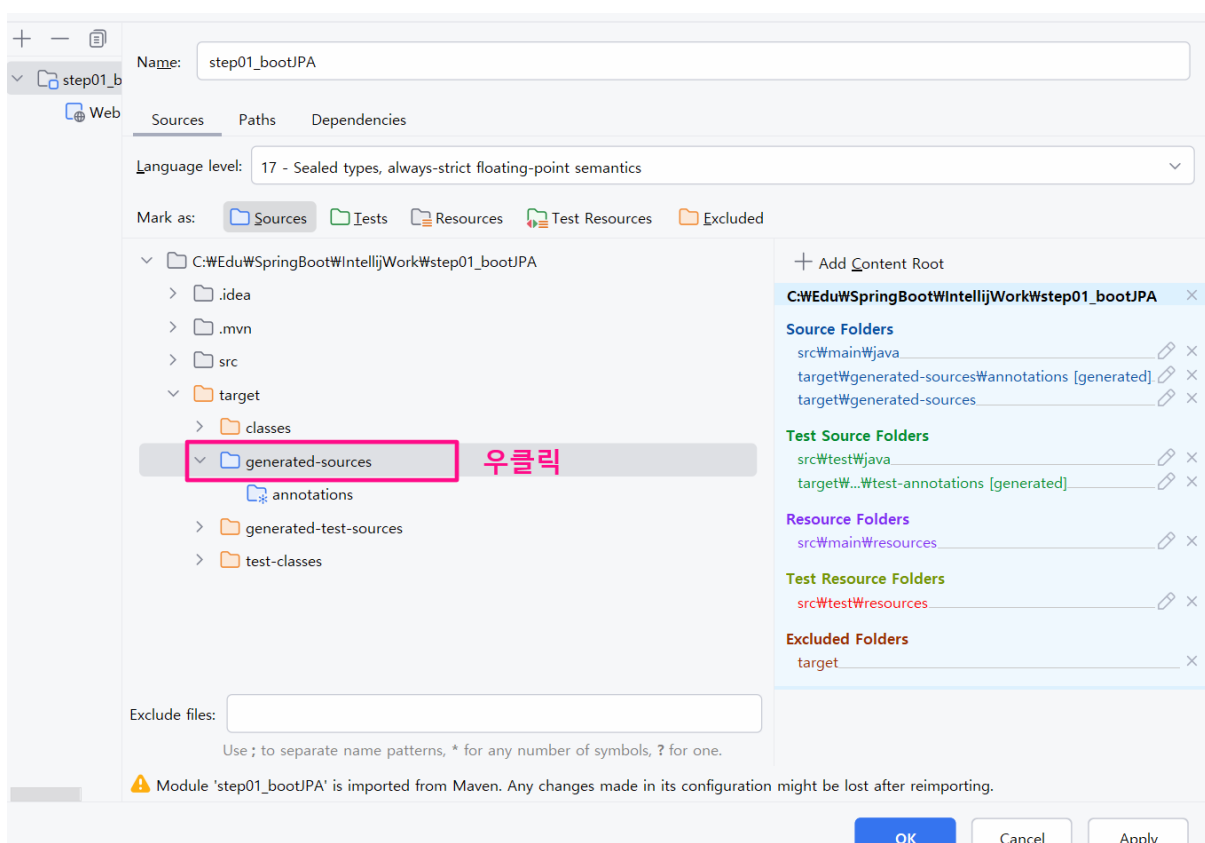
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
 Hibernate: select b1_0.bno,b1_0.content,b1_0.insert_date,b1_0.title,b1_0.update_date,b1_0.writer from board b1_0 where b1_0.bno>? or b1_0.title=?
 2024-11-03T13:20:13.777+09:00 INFO 15076 --- [sten01 boot.JPA] [ionShutdownHook] i.LocalContainerEntityManagerFactoryBean

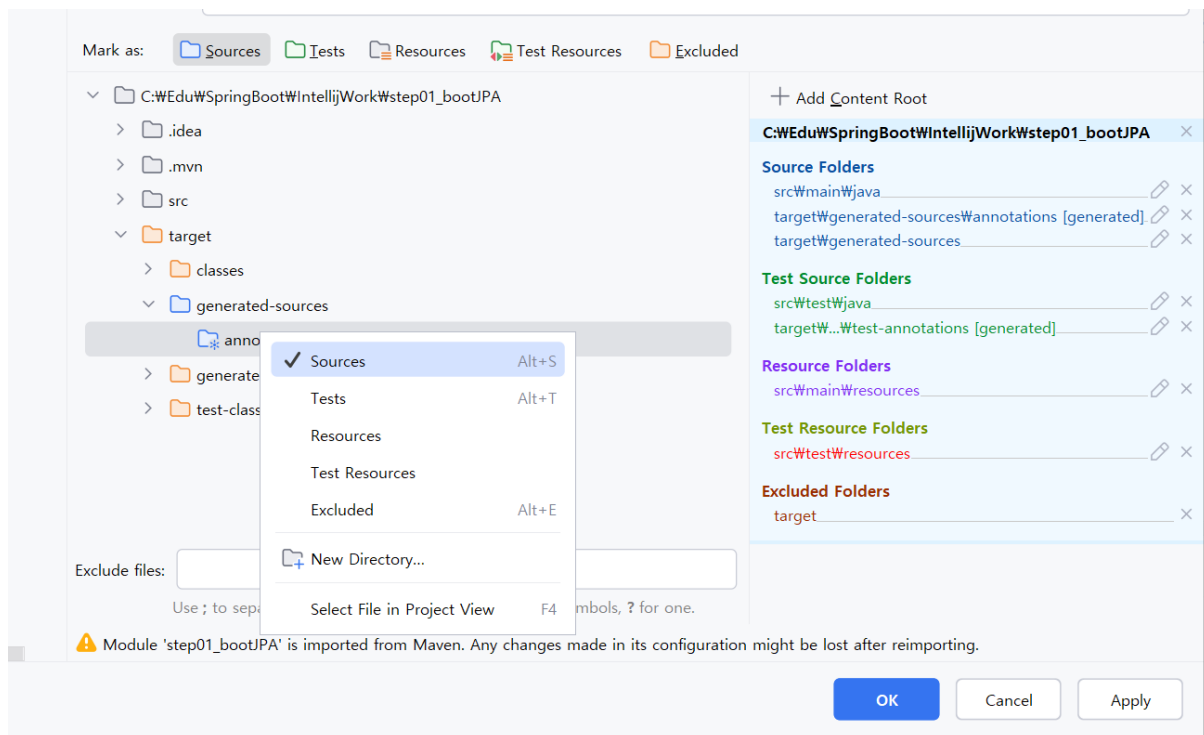
QueryPredicateExecutor<> interface 사용하기

- * interface에 QueryPredicateExecutor<> 상속받는다.
- * - QueryPredicateExecutor안에서 제공하는 메소드를 사용해서 자바중심으로 조건을 만들수 있다. (자동으로 QEntity이름.java 가 생성된다)
- * ex) ~.findAll(Predicate p)

QueryDSL 참고

http://querydsl.com/static/querydsl/4.0.1/reference/ko-KR/html_single/





```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>${maven-compiler-plugin.version}</version> <configuration>
  <annotationProcessorPaths>
    <path>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>${lombok.version}</version> </path>
    <path>
      <groupId>com.querydsl</groupId>
      <artifactId>querydsl-apt</artifactId>
      <version>${querydsl.version}</version>
      <classifier>jakarta</classifier>
    </path>
    <path>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>${jakarta-persistence-api.version}</version>
    </path>
  </annotationProcessorPaths>
  </configuration>
</plugin>

```

```

    </path>
  </annotationProcessorPaths>
  <generatedSourcesDirectory>${project.build.directory}/generated-
sources/annotations</generatedSourcesDirectory>
</configuration>
</plugin>

```

